

Lightator: An Optical Near-Sensor Accelerator with Compressive Acquisition Enabling Versatile Image Processing

Mehrdad Morsali[†], Brendan Reidy[‡], Deniz Najafi[†], Sepehr Tabrizchi[§], Mohsen Imani^{*},
Mahdi Nikdast^{**}, Arman Roohi[§], Ramtin Zand[‡], and Shaahin Angizi[†]

[†] Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ, USA

[‡] Department of Computer Science and Engineering, University of South Carolina, Columbia, SC, USA

[§] School of Computing, University of Nebraska–Lincoln, Lincoln, NE, USA

^{*} Department of Computer Science, University of California Irvine, Irvine, CA, USA

^{**} Department of Electrical and Computer Engineering, Colorado State University, Fort Collins, CO, USA

m.imani@uci.edu, mahdi.nikdast@colostate.edu, aroohi@unl.edu, ramtin@cse.sc.edu, shaahin.angizi@njit.edu

ABSTRACT

This paper proposes a high-performance and energy-efficient optical near-sensor accelerator for vision applications, called Lightator. Harnessing the promising efficiency offered by photonic devices, Lightator features innovative compressive acquisition of input frames and fine-grained convolution operations for low-power and versatile image processing at the edge for the first time. This will substantially diminish the energy consumption and latency of conversion, transmission, and processing within the established cloud-centric architecture as well as recently designed edge accelerators. Our device-to-architecture simulation results show that with favorable accuracy, Lightator achieves 84.4 Kilo FPS/W and reduces power consumption by a factor of $\sim 24\times$ and $73\times$ on average compared with existing photonic accelerators and GPU baseline.

CCS CONCEPTS

• **Hardware** → **Emerging optical and photonic technologies**.
ACM Reference Format:

Mehrdad Morsali, Brendan Reidy, Deniz Najafi, Sepehr Tabrizchi, Mohsen Imani, Mahdi Nikdast, Arman Roohi, Ramtin Zand, and Shaahin Angizi. 2024. Lightator: An Optical Near-Sensor Accelerator with Compressive Acquisition Enabling Versatile Image Processing. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3656261>

1 INTRODUCTION

While the prevalence of the Internet of Things (IoT) has grown significantly, it still lacks inherent intelligence and heavily depends on cloud-based decision-making. In such a cloud-oriented paradigm, a considerable portion of data created by IoT sensors remains unprocessed [15, 23]. Vision sensors typically capture light and convert it into electrical signals, which are subsequently stored, processed, transmitted, and utilized. This procedure necessitates the transformation of all individual pixels into predetermined digital values with a fixed bit-width (e.g., 8 bits [3, 23]). Remarkably, the major share of power consumption in traditional vision sensors, exceeding 96% [23], is ascribed to the conversion and retention of

pixel values. This is predominantly associated with memory- and computation-intensive algorithms and the limited processing capabilities of current IoT devices, which are constrained by power and size limits [3, 21]. To confront these challenges, a shift from a cloud-oriented to a thing-centered (data-centric) approach is imperative, wherein IoT nodes locally process the data [10].

Recent efforts to improve CMOS image sensors for faster processing of Deep Neural Network (DNN) workloads have resulted in emerging new solutions such as integrating sensors and processors on a single chip, known as Processing-Near-Sensor (PNS) [5, 11, 25]; And incorporating computation units with individual pixels, termed Processing-In-Sensor (PIS) [3, 20, 23, 24]. The PIS platform processes pre-Analog-to-Digital Converter (pre-ADC) data before transmitting it to the on/off-chip processor. However, challenges remain, such as high energy usage in ADC, DAC, and sense amplifiers, limiting the deployment of all DNN layers into the pixel array [9, 15]. Most studies have focused on accelerating the initial layer and outsourcing the remaining layers to a digital accelerator due to the restricted resources of PIS. Therefore, three key challenges remain unaddressed in current electronic PIS/PNS designs: (i) power-hungry peripherals and ADC/DAC units, even when reduced for sensing and computing [8, 10, 16, 23]; (ii) significant area overhead and power consumption in recent PNS/PIS units, necessitating additional memory for intermediate data storage [3, 15, 20]; and (iii) constrained computation speed due to electronic systems operating at a few GHz, lacking the capability to support the high speeds and extensive parallelism observed in optical systems with photo-detection rates exceeding 100GHz [7, 16, 19].

With further advancement of integrated photonic devices (e.g., energy-efficient and tunable Microring Resonators (MRs) and Mach-Zehnder modulators), CMOS-compatible silicon photonics offer a promising alternative to digital electronics for high-speed and energy-efficient optical DNN accelerators, as evidenced by various research studies [12, 14, 16, 19, 27], though the edge deployment of such MR devices has been insufficiently explored. Besides, even the existing MR-based accelerators have faced several challenges that this work aims to solve including (i) excessive use and tuning power overhead of MRs in accelerators for activation parameters [16, 19]; (ii) high power and area overhead resulting from excessive using of ADCs and DACs [12, 14, 17]; (iii) limited flexibility in processing various DNN layers (Pooling, etc.) with no compression support; and (iv) lack of correlated hardware mapping methodologies to support various kernel sizes in DNNs. The key contributions of this work are as follows: (1) we propose a high-performance

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0601-1/24/06.

<https://doi.org/10.1145/3649329.3656261>

and energy-efficient optical PNS accelerator for vision applications called Lighttator that can fully process various DNN layers with weight-based optical cores without relying on the cloud; (2) to meet the physical limitation of the photonic domain and power budget of IoT devices, we create innovative microarchitectural and circuit-level strategies for Lighttator that enables compressive acquisition of input frames and novel hardware partitioning and mapping mechanisms to support various DNN kernel sizes; (3) we establish a solid device-to-architecture evaluation framework from the ground up and conduct thorough performance analysis and comparison of our proposed designs with state-of-the-art optical and electronic accelerator designs.

2 BACKGROUND AND RELATED WORK

Offering notably elevated operational bandwidth compared to electronic accelerators along with addressing fan-in/fan-out problems make silicon-photonic-based accelerators a promising candidate to accelerate DNN and machine vision applications [12, 16, 18, 27]. Such accelerators can be broadly categorized into two primary designs: *coherent* and *non-coherent* architectures. Within the coherent category, a single wavelength is employed for operations, and weight/activation parameters are incorporated into the electrical field amplitude, phase, or polarization of an optical signal [26]. Conversely, the non-coherent designs [16, 19] employ multiple wavelengths each of which capable of conducting computations concurrently. Within non-coherent architectures, the weight and input parameters of DNN are imprinted upon the signal’s amplitude [16, 19]. To manipulate individual wavelengths, MRs—depicted in Fig. 1—can be employed whose central frequency can be actively adjusted (i.e., through tuning mechanisms using, e.g., microheaters or PIN junctions), to selectively interact with specific wavelengths. By appropriately tuning the MRs, the incoming light intensity of a specific wavelength can be weighted. In non-coherent designs [16, 19], MRs as a fundamental component hold the weight and activation values to be utilized in the Multiply-and-ACcumulate (MAC) operation. During photonic MAC, the transmission spectrum of input lights can be multiplied by the value adjusted on the MRs (through applying a tuning signal, see Fig. 1). Such a value is adjusted by tuning the resonant wavelength of the MR which can partially overlap with the wavelength of the input signal, to imprint the parameter into the transmission spectrum of the input signal (see Fig. 1). The resonant wavelength is given by $\lambda_{res} = \frac{n_{eff} \times L}{m}$, where n_{eff} is the effective refractive index of the MR, and L and m denote MR’s circumference and order of the resonant mode. [4].

Previous studies have explored accelerating DNNs through the application of both coherent and non-coherent photonic principles. LightBulb [27] as a fully binarized Convolutional Neural Network (CNN) accelerator has been proposed which replaces the floating-point MAC operations with photonic XNOR and popcounts. With reduced computation latency and memory storage, LightBulb’s excessive ADCs increased the power consumption of the design. Robin [19] also presents an MR-based binary CNN accelerator, optimizing electro-optic components across device, circuit, and architecture layers. Despite circuit-level tuning enhancements to reduce inference latency, the excessive number of MRs and subsequent DACs required for the tuning process reduced the efficiency of the design. CrossLight [16] as a 4-bit weight-input CNN accelerator

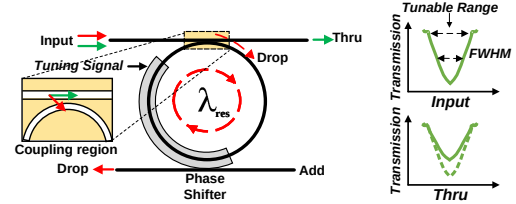


Figure 1: MR input and through ports’ spectra after imprinting a parameter (using tuning signal). By adjusting the MR’s resonant wavelength (λ_{res}) using the phase shifter, part of the input signal drops into the ring (through the coupling region) towards the drop port while the remaining propagates towards the through port, hence imprinting any parameter in the transmitted signals. FWHM is full width at half maximum of resonance spectrum.

requires tuning both activation and weight values in the MRs and only supports convolution layer processing similar to the previous designs. The design in [17] proposes a CNN accelerator with mixed-precision weight-input support. This non-coherent silicon photonic accelerator utilizes both Wavelength-Division Multiplexing (WDM) and Time-Division Multiplexing (TDM). However, the persistent use of DACs and ADCs as inter-layer transformers is a notable concern which increased the overall area and power consumption of the entire architecture. HolyLight [12] as a nanophotonic accelerator enhances the inference throughput of CNN by using MR-based adders and shifters instead of ADCs. Nevertheless, over-utilization of MRs for both activation and weight values not only increased overall delay and power consumption but also reduced its flexibility to be used for various DNNs.

3 LIGHTTATOR ARCHITECTURE

We propose Lighttator as a high-performance, energy-efficient, and versatile PNS accelerator with compressive acquisition for real-time image processing at the edge. The key idea behind developing such an architecture is to have a standalone optical framework (not relying on off-chip processors [3, 20, 23]) for the first time to compress and process all layers in Multi-Layer Perceptron (MLPs) and CNNs in a low-bit-width fashion to tailor the trade-offs between the power consumption and accuracy. The high-level operational flow of Lighttator represented by node i in a multi-node IoT structure is shown in Fig. 2. The design consists of a $m \times n$ sensor array and an ultra-fast Optical Core (OC) interfacing through a Directly-Modulated VCSEL Array (DMVA). In step ①, the input frame f_i is captured by a global-shutter RGB image sensor and processed in an innovative ADC-less fashion with the DMVA unit. In ②, the resulting waveguides can be optionally fed to an OC’s Compressive Acquisitor (CA) unit that reduces the spatial dimension by mean pooling across channels and strided convolution to generate f_j . This step can be readily skipped depending on the workload and requirements. In ③, the All-in-One Convolver (AOC) processes the DNN layer and transmits the results f_o in step ④ to be used by DMVA as the input to the next layer. Therefore, step ③↔④ features layer-by-layer DNN process through a novel hardware mechanism to reuse DMVA and eliminates the need for conventional area-/power-consuming activation banks [16, 19] and prepares the result for transmitter ⑤.

The detailed architecture of Lighttator is presented in Fig. 3. A sensor array in an ADC-less fashion is connected to the VCSEL driver circuit using a Comparator-based pixel Reading Circuit (CRC).

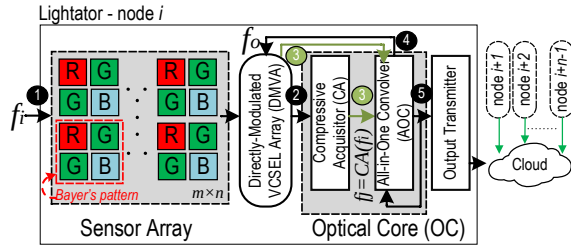


Figure 2: High-level operational flow of Lightator.

VCSEL driver drives an array of VCSELs in the OC. The Matrix-Vector Multiplication (MVM) banks and the subsequent summation section handle the execution of MAC operations across different network layers. The primary advantage of Lightator processing core is that it only requires mapping weight data onto MRs, while activation values are directly modulated onto the core’s input light through VCSELs by adjusting their driving currents, unlike prior designs discussed in the previous section. This configuration allows the entire capacity of the OC to be dedicated to weight values rather than activation, resulting in substantial energy savings, as driving VCSELs consumes significantly less power compared to tuning MRs [16, 19]. Moreover, additional energy efficiency is achieved as the VCSEL driver directly takes the digital output of the previous layer, eliminating the need for conversion to analog MR tuning signals using DACs. The electronic component on top consists of the activation functions supporting *Sign*, *ReLU*, and *tanh*, as well as the storage for the weights and the activated feature maps from previous layers. A more detailed explanation of the various components of Lightator architecture is provided in the following.

ADC-Less Imager. A 256×256 global-shutter RGB image sensor has been considered in the presented design. Every pixel's Photo-Diode (PD) generates a photo-current with respect to the external light intensity which in turn leads to a voltage drop (V_{PD}). By utilizing the CRC, the usage of power-hungry and area-consuming ADCs is resolved. The CRC is responsible for reading the output of the pixel and so the analog output of pixels will be converted to 4-bit digital data.

Directly-Modulated VCSEL Array. DMVA is developed to convert its electrical input to light with a specific wavelength and intensity. Instead of generating raw light, the intensity of light generated by VCSELs is correlated with the input data of the VCSEL driver, and the wavelength is correlated with the VCSEL structure itself. The input of the VCSEL driver comes from either the pixel

array or the output of the previous layer which is processed by the OC. This input is modulated to a specific wavelength and fed to OC as activation to participate in the MAC operation of the next DNN layer. The DMVA consists of three components as shown in Fig. 4: CRC, Selector, and VCSEL driver. Each CRC unit (Fig. 4(a)) contains 15 voltage comparators and is utilized instead of ADCs to read the pixel's output voltage. CRC receives pixel's V_{PD} and compares it with 15 reference voltages (V_{Ref}) which are spanned in the range of pixel output voltage. According to the value of the V_{PD} , the output of the comparators (V_S) will be either '0' or '1' and later these binary voltages will be used to control the VCSEL's driving transistors. Fig. 4(d), depicts a sample waveform of the pixel's output voltage and comparator outputs that are used for controlling the driving current of the VCSELs. According to Fig. 4(d), by increasing the V_{PD} , more number of comparators outputs (V_S) will be '1' leading larger number of ON transistors in the VCSEL driver circuit.

A selector circuit is used to select the input-controlling voltages of the VCSEL driver as depicted in Fig. 4(b). During processing the first layer of the network, the selector connects the output of the pixel array to VCSEL and later when the rest of the layers are getting processed, the selector connects the output of previous network layers as the input of VCSEL driver to be modulated and fed as the activation of next layers. The VCSEL driver circuit (Fig. 4(c)) comprises 16 parallel driving transistors that encode 4-bit data. Depending on input signals from either the CRC (V_S) or the output of the previous layer (V_B) coming from the selector in Fig. 4(b), the number of transistors supplying VCSEL's driving current will be adjusted. When pixel voltage is large or the digital input from the previous layer is greater, more driving transistors will be activated, leading to an increase in the light intensity generated by VCSEL.

Optical Core. OC’s MR-based computational units are virtually divided into multiple banks as color-coded in Fig. 3 including compressive acquirer, convolutional layer, and fully-connected layer to execute various DNNs all through adjusting weight parameters and MVM operation if required.

1. *All-in-One Convolver (AOC)*: The OC comprises three main components as depicted in Fig. 3, VCSELs, MVM banks, and the summation section. VCSELs generate light waves that represent activation values, with the intensity of the light corresponding to these values. MVM banks contain MRs that are mapped with weight values and partitioned in the arms. The MRs adjust the intensity of incoming light based on their mapped weight values, affecting only light with the same wavelength as the MR. This

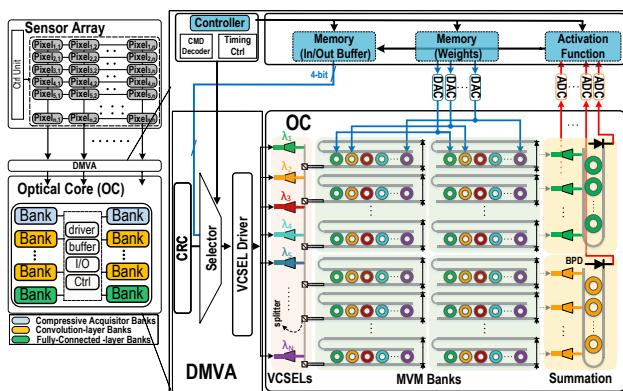


Figure 3: Lightator architecture consisting of a sensor array and the optical core.

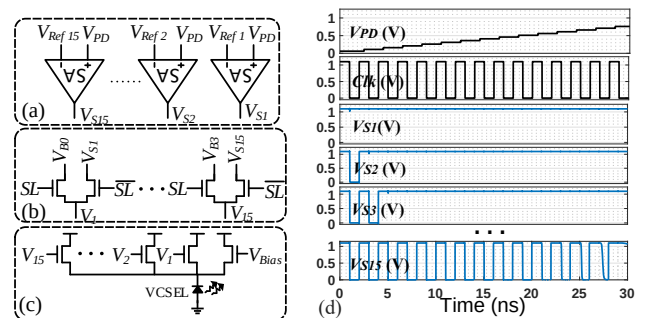


Figure 4: Components of the DMVA: (a) CRC, (b) Selector, (c) VCSEL driver, (d) Sample waveforms of CRC input from the pixel and respective outputs .

process involves multiplying the activation's light intensity with the weight stored in the MR that is shown in Fig. 5. To perform MAC operation, a light signal containing all of the required activation values that are modulated on different wavelengths passes through the arm housing MRs with mapped weights. As the light passes the arm, each MR influences the intensity of light at a wavelength corresponding to that specific MR. A Balanced PhotoDetectors (BPD) at the end of each arm handles accumulation, enabling MAC operations to be performed in each arm of the MVM bank.

The number of multiplication that can be conducted inside an arm is equal to the number of MRs in the arm. In the case of processing fully connected layers or convolutional layers with large kernel sizes that require MAC operation of a large number of activations and weights, the number of multiplications exceeds the capacity of the arm. In these cases, a large number of MACs are divided into smaller segments that can fit within an arm. Subsequently, these segmented MAC results are summed in the summation section to obtain the final MAC result. To facilitate the processing of multiple layers and enable the processing of the entire neural network on the Lightator platform, in addition to the optical core, an electronic part is required. This part is essential for storing the weight values of various layers since, due to the core's physical limitations, all of the weights of a network cannot be simultaneously mapped to the optical core. Thus, weight values are stored in a dedicated memory and then mapped to the MRs during the processing of each layer. Another memory is utilized to retain the processed output from the network's previous layer, which is subsequently fed as activation to the next layer. In addition, implementing an activation function at the end of each layer is more efficient in the electronic domain than the optic domain [16, 19], thus, the electronic part is responsible for performing the activation function. The controller unit controls the procedure and timing of the platform.

2. Compressive Acquisitor (CA): CA banks are dedicated to serving as a compression/pooling layer, where an RGB-to-grayscale conversion and/or configurable average pooling can be done all through adjusting MRs. We propose to conduct the compression in a single operational cycle by mapping proper compression weights to the OC banks and performing the corresponding MAC operation. The conversion from RGB to grayscale can be achieved by forming a weighted sum of the R, G, and B pixel values after CRC as $P_{Grayscale} = (0.299 \times P_R) + (0.587 \times P_G) + (0.114 \times P_B)$. And, as an example, the 2×2 average pooling layer containing P_1 to P_4 pixels can be formulated similar to a weighted sum as follows: $P_{Avg} = (0.25 \times P_1) + (0.25 \times P_2) + (0.25 \times P_3) + (0.25 \times P_4)$. Therefore, a nicely-compressed and gray-scale-converted input can be given by properly tuning the weight parameters as follows.

$$P_{AvgGray} = (0.25 \times 0.299 \times P_{1R}) + (0.25 \times 0.587 \times P_{1G}) + (0.25 \times 0.114 \times P_{1B}) + \dots + (0.25 \times 0.299 \times P_{4R}) + (0.25 \times 0.587 \times P_{4G}) + (0.25 \times 0.114 \times P_{4B}) \quad (1)$$

Where in P_{ij} , i is the pixel number identifier and j denotes the channel which can be R, G, or, B. By using the above method and mapping the coefficients of the resultant equation (1) in the OC's MR banks, RGB-to-grayscale conversion and average pooling of any size can be conducted simultaneously.

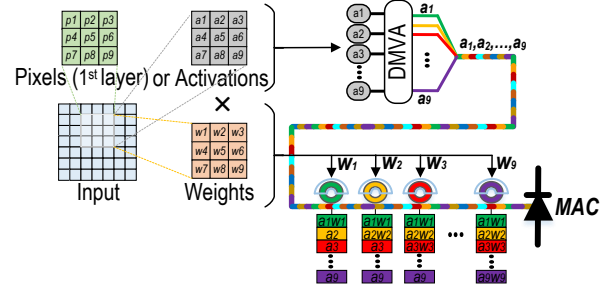


Figure 5: Implementing a 3×3 kernel in an arm.

4 HARDWARE MAPPING

Methodology. The MVM banks within the OC are vital for processing various layers like compression, convolution, or fully connected layers, requiring respective weights assigned to MRs. In our setup, MRs are grouped in sets of 9 within each arm to efficiently handle 3×3 kernel strides, resulting in 96 banks organized into an 8×12 array, with each bank containing 54 MRs. Thus, the MVM banks collectively accommodate 5184 MRs, allowing a maximum of 5184 MAC operations per operational cycle of the OC. Fig. 6 depicts the bank configuration for MAC operations in a convolutional layer with 3×3 , 5×5 , and 7×7 kernel sizes, with summation sections located at the right end of each bank. As the configuration of OC is specified for 3×3 kernels, as depicted in Fig. 6(a), all of the MRs are allocated to be mapped with weight values enabling each arm to execute a stride. BPD performs the required summation, enabling direct transmission of the MAC result without using the summation component, which remains inactive (depicted in gray in Fig. 6(a)). This way, each bank can execute 6 strides. For 5×5 kernels, 25 weight values are mapped on MRs, with 3 arms per bank allocated for one stride. 27 MRs are available in 3 arms, thus, in each set of arms, 2 MRs remain unused and inactive, indicated by the gray shading in Fig. 6(b). Since each arm lacks summation capability for all multiplication elements, additional summation stages for partial sums are required, with the initial stage activated while the second stage remains inactive (grayed out). As illustrated in Fig. 6(b), in this case, each bank can perform 2 strides. For a 7×7 kernel size, a total of 49 MRs are necessary for weight mapping, leading to the entire bank being dedicated to a single stride. Nevertheless, 5 MRs per bank remain inactive and unused, shown in gray in Fig. 6(c). Through further processing involving two stages of the summation part, partial products are combined, allowing the final MAC results to be sent out. In the case of fully-connected layers, we segment the entire MAC operations into sets of 9 MACs, map their corresponding weights to arms, and subsequently aggregate the partial results using the summation part to derive the ultimate MAC result.

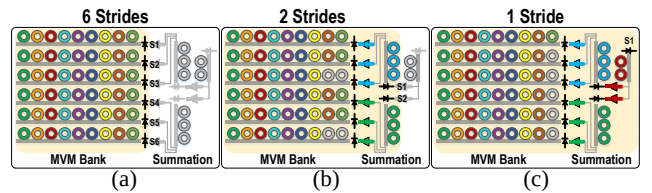


Figure 6: Hardware mapping for (a) 6 Strides (3×3), (b) 2 Strides (5×5), (c) 1 Stride (7×7).

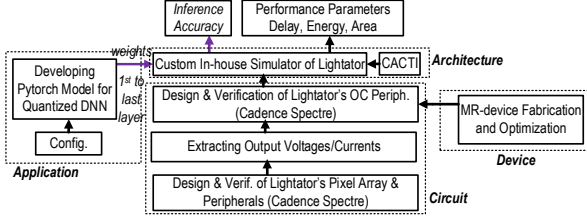


Figure 7: Proposed bottom-up evaluation framework.

5 EXPERIMENTS

Framework. As shown in Fig. 7, the assessment framework consists of device-, circuit-, architecture-, and application-level components. At the device level, we manufactured and fine-tuned the MR devices and obtained the circuit parameters for co-simulation with interface CMOS circuits in Cadence Spectre and SPICE. Progressing to the circuit level, we initially implement the pixel’s array and peripheral circuitry using the 45nm NCSU Product Development Kit (PDK) library [1] in Cadence, from which we derive the output voltages and currents. Then we proceed to develop all Lightator’s components excluding kernel banks (implemented in Cacti [22]) in Cadence Spectre. At the application level, we train PyTorch models w.r.t. the under-test DNN models and datasets and extract weight parameters. These parameters are then quantized and mapped into the OC for adjusting MR elements. To preserve optimal accuracy post-precision reduction, we undertake an additional six epochs of training employing quantization-aware techniques. This ensures the model’s robustness and performance integrity in the face of reduced precision. At the architecture level, we develop a custom in-house simulator for Lightator to work with the 1st-to-last layer weight parameters and calculate both the execution time and power consumption required for the DNN models as well as inference accuracy. Moreover, it offers flexibility in terms of MVM array configuration and the selection of peripheral designs. We conduct experiments on Lightator considering various [Weight: Activation] configurations with several datasets, including MNIST evaluated on LeNET, and CIFAR10, and CIFAR100 on VGG9.

Power Consumption & Performance. Fig. 8 shows the layer-wise breakdown of components of power consumption including ADCs, DACs, DMVA (with CRC, VCSELs, and drivers), Tuning circuitry (TUN), BPDs, and Misc. (Controller, etc.) for LeNET model mapped to Lightator. Lightator effortlessly implements all convolutional and pooling layers (indicated by L_i) for three weight and activation [W:A] configurations of [4:4], [3:4], and [2:4]. Pooling layers are implemented within CA banks with pre-set weight coefficients. We observe that decreasing the bit-width of weight parameters for each layer results in power saving for the edge device, where on

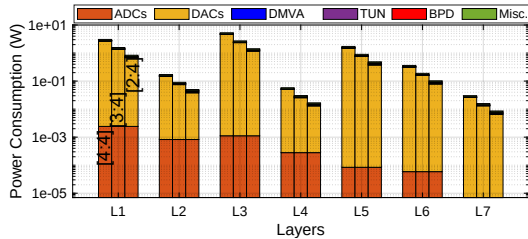


Figure 8: Break-down of power consumption for LeNET on [4:4], [3:4], and [2:4]. Note: Pooling layers are implemented within CA banks with pre-set weight coefficients.

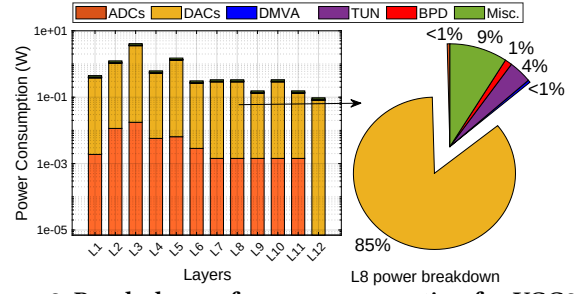


Figure 9: Break-down of power consumption for VGG9 on [3:4] configuration.

average 2.4 $\times$ more power efficiency is reported. This mainly comes from power-gating parts of the 4-bit DAC circuits that are related to its extra bit precision, when they process 3-bit and 2-bit data. In Fig. 9, the distribution of power consumption components for the VGG9 model is depicted layer-by-layer, specifically focusing on configurations limited to [3:4]. We leverage CA banks for a light compression of input images as the proof-of-concept before feeding them into the model. This leads to a 42.2% reduction in power consumption of the first layer. The pie chart in Fig. 9 clarifies the breakdown of power consumption in a sample layer as well. We observe that consistently across all layers, DACs contribute to more than 85% of the total power consumption, as DAC usage is required to convert all of the weight values to analog inputs for tuning purposes.

Comparison with Optical Accelerators. Table 1 provides our comprehensive simulation results for selected MR-based optical accelerators and Lightator in various [W:A] configurations compared with the baseline, an NVIDIA Geforce RTX 3060Ti GPU. The under-test DNN accelerators includes LightBulb [27], HolyLight [12], HQNNA [17], Robin [19], and CrossLight [16] discussed in the background section. To ensure an unbiased assessment, we created the designs from the ground up resembling the original design, employing the evaluation framework and our in-house simulator, and reported the results in a reasonable area constraint for all accelerators ($\sim 20\text{-}60\text{mm}^2$). Our framework features 96 banks, each comprising 6 arms with 9 MRs.

Table 1: Performance comparison with optical designs.

Designs & [W:A]	Process node (nm)	Max Power (W)	KFPS/W	Accuracy (%)		
				MNIST	CIFAR10	CIFAR100
baseline [32:32] [§]	8	200	-	98.53	90.46	67.8
LightBulb [1:1] [27]	32	68.3	57.75	96.7	-	-
HolyLight [4:4] [12]	32	66.9	3.3	98.9	88.5	-
HQNNA [17]	45	-	34.6	-	89.68	61.95
Robin [1:4] [19]	45	106	46.5	-	62.5	45.6
CrossLight [4:4] [16]	-*	84-390	10.78-52.59	92.6	78.85	-
Lightator [4:4]	45	5.28	61.61	98.12	88.87	64.22
Lightator [3:4]	45	2.71	117.65	98.05	86.3	61.04
Lightator [2:4]	45	1.46	188.24	93.95	70.55	41.4
Lightator-MX [4:4][3:4] [†]	45	3.64	84.4	97.85	85.65	63.37
Lightator-MX [4:4][2:4] [‡]	45	1.97	126.6	94.8	78.87	51.29

[§] NVIDIA Geforce RTX 3060Ti GPU. * Data is not reported/not achievable in the paper. [†] Lightator with mixed-precision scheme, where L1[4:4] - L2:LN [3:4]. [‡] Lightator with mixed-precision scheme, where L1[4:4] - L2:LN [2:4].

Here we list our key observations. (1) We observe Lightator’s variants demonstrate remarkable power efficiency over counterpart designs on the VGG9 model running CIFAR100, e.g., Lightator [3:4] consumes 2.71 W which can be drawn from the low power budget of edge devices, however, the best low-power accelerator, i.e., HolyLight [12] requires 66.9 W or higher [19]. Such striking power efficiency comes from (i) eliminating the MRs tuned by activation parameters which results in saving the tuning power required for the MRs, and (ii) reducing the additional power and area requirements caused by the extensive utilization of ADCs and DACs. (2)

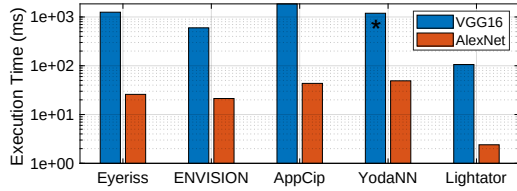


Figure 10: Log-scaled execution time of various accelerators. YodaNN’s results for VGG16 are substituted with VGG13.

On average the Lightator reduces power consumption by $\sim 73\times$, $24.68\times$, $30.9\times$ compared with the baseline [32:32], HolyLight [4:4] [12], and CrossLight [4:4] [16], respectively. (3) As we reduce the weight bit-width, the power consumption can be reduced at the cost of accuracy degradation, where Lightator [3:4] achieves $\sim 2\times$ power saving at the cost of 3.17% accuracy drop. (4) The mixed precision implementation of DNNs on Lightator is termed as Lightator-MX in Table 1 in which the first layer configuration is kept to [4:4] and the rest of the layers are processed in [3:4] or [2:4] precision. We observe the trade-offs between power consumption and accuracy that can be readily adjusted based on the image-processing task requirements. As shown, Lightator-MX [4:4][3:4] as an optimal design imposes ~ 0.9 W extra power consumption to the Lightator [3:4] increasing the accuracy of CIFAR100 by 2.33%. (5) As for the throughput ($\frac{\text{frame}}{\text{second}}$) per watt, Lightator [3:4] demonstrates 117.65 kilo FPS/W increasing inference performance by $\sim 2\times$ compared to the best result reported for LightBulb [27]. Overall, considering the test accuracy results over three under-test data-sets, Lightator-MX [4:4][3:4] offers the best performance-quality number with 84.4 kilo FPS/W. (6) As for only accuracy, our experiments generally reveal that Lightator with [3:4] and [4:4] configurations could demonstrate acceptable accuracy over three under-test data-sets. On MNIST and CIFAR10 data-sets, Lightator [4:4] achieves the second-highest accuracy among all optical accelerators after HolyLight [12] and HQNNA [17], respectively, while showing higher KFPS/W compared to them. (7) We observe that activations and weights exhibit increasing sensitivity to changes in bit-width.

Comparison with Electronic Accelerators. To demonstrate the intrinsic parallelism observed in Lightator as an optical accelerator, we further explore its execution time compared with four well-known digital electronic accelerators, each with a distinct parallelism technique and hardware mapping method, i.e., Eyeriss [6], YodaNN [2], AppCip [20], and ENVISION [13] running VGG16 and AlexNet. Eyeriss employs a spatial architecture that utilizes row-stationary dataflow to minimize energy consumption. YodaNN is an ASIC accelerator optimized for binary-weight CNNs with support for different filter sizes in parallel. AppCip as a PIS implements instant RGB-to-grayscale conversion, highly parallel analog convolution-in-pixel, and low-precision quinary weight neural networks. ENVISION utilizes subword parallel MACs with dynamic adjustments to voltage, frequency, and bit precision scaling. The simulation results plotted in Fig. 10 demonstrate the superiority of the optical accelerator in processing DNN layers compared with the electronic ones over both models. We observe Lightator reduces the execution time by a factor of $10.7\times$, $20.4\times$, $18.1\times$, $8.8\times$ over Eyeriss [6], YodaNN [2], AppCip [20], and ENVISION [13] on AlexNet, respectively. A similar trend is observable for the VGG16 model.

6 CONCLUSION

Here, we presented an efficient optical near-sensor accelerator for vision applications named Lightator. Our design features innovative

compressive acquisition of input frames and fine-grained convolution operations for low-power and versatile image processing at the edge. Our results demonstrate that with acceptable accuracy, Lightator achieves 84.4 Kilo FPS/W and reduces power consumption by a factor of $\sim 24\times$ and $73\times$ on average compared with recent photonic accelerators and GPU baseline.

ACKNOWLEDGMENT

This work is supported in part by the National Science Foundation (NSF) under grant no. 2228028, 2216772, 2247156, 2046226, 2167773, 2303114, 2127780, 2319198, 2321840, 2312517, 2235472, Semiconductor Research Corporation (SRC), ONR Young Investigator Program Award, ONR #N00014-21-1-2225 and #N00014-22-1-2067, and the Air Force Office of Scientific Research under award #FA9550-22-1-0253.

REFERENCES

- [1] 2011. *NCSU EDA FreePDK45*. <http://www.eda.ncsu.edu/wiki/FreePDK45>
- [2] Renzo Andri et al. 2018. Yodann: An architecture for ultralow power binary-weight cnn acceleration. *IEEE TCAD* 37 (2018), 48–60.
- [3] Shaahin Angizi et al. 2023. PISA: A Non-Volatile Processing-In-Sensor Accelerator for Imaging Systems. *IEEE TETC* (2023).
- [4] Wim Bogaerts et al. 2012. Silicon microring resonators. *Laser & Photonics Reviews* (2012), 47–73.
- [5] Stephen J Carey et al. 2013. A 100,000 fps vision sensor with embedded 535GOPS/W 256× 256 SIMD processor array. In *Symposium on VLSI IEEE*.
- [6] Yu-Hsin Chen et al. 2017. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE JSSC* 52 (2017).
- [7] Qixiang Cheng et al. 2020. Silicon photonics codesign for deep learning. *Proc. IEEE* 108 (2020), 1261–1282.
- [8] Jaehyuk Choi et al. 2015. An energy/illumination-adaptive CMOS image sensor with reconfigurable modes of operations. *IEEE JSSC* 50, 6 (2015), 1438–1450.
- [9] Abbas El Gamal et al. 1999. Pixel-level processing: why, what, and how?. In *Sensors, Cameras, and Applications for Digital Photography*, Vol. 3650. SPIE, 2–13.
- [10] Tzu-Hsiang Hsu et al. 2019. AI edge devices using computing-in-memory and processing-in-sensor: from system to device. In *IEDM*.
- [11] Tzu-Hsiang Hsu, Yi-Ren Chen, et al. 2020. A 0.5-V Real-Time Computational CMOS Image Sensor With Programmable Kernel for Feature Extraction. *IEEE JSSC* 56 (2020), 1588–1596.
- [12] Weichen Liu et al. 2019. HolyLight: A nanophotonic accelerator for deep learning in data centers. In *DATE, IEEE*, 1483–1488.
- [13] Bert Moons et al. 2017. 14.5 envision: A 0.26-to-10tops/w subword-parallel dynamic-voltage-accuracy-frequency-scalable convolutional neural network processor in 28nm fdsoi. In *ISSCC, IEEE*, 246–247.
- [14] Kyle Shiflett et al. 2021. Albireo: Energy-efficient acceleration of convolutional neural networks via silicon photonics. In *ISCA, IEEE*, 860–873.
- [15] Ruibing Song et al. 2022. A reconfigurable convolution-in-pixel cmos image sensor architecture. *IEEE TCSVT* (2022).
- [16] Febin Sunny et al. 2021. CrossLight: A cross-layer optimized silicon photonic neural network accelerator. In *DAC, IEEE*, 1069–1074.
- [17] Febin Sunny et al. 2022. A silicon photonic accelerator for convolutional neural networks with heterogeneous quantization. In *GLSVLSI*, 367–371.
- [18] Febin P Sunny et al. 2021. ARXON: A framework for approximate communication over photonic networks-on-chip. *TVLSI* 29 (2021), 1206–1219.
- [19] Febin P Sunny et al. 2021. ROBIN: A robust optical binary neural network accelerator. *ACM TECS* 5s (2021), 1–24.
- [20] Sepehr Tabrizchi et al. 2023. AppCIP: Energy-Efficient Approximate Convolution-in-Pixel Scheme for Neural Network Acceleration. *IEEE JETCAS* (2023), 225–236.
- [21] Kea-Tiong Tang et al. 2019. Considerations of integrating computing-in-memory and processing-in-sensor into convolutional neural network accelerators for low-power edge devices. In *Symposium on VLSI IEEE*.
- [22] Shyamkumar Thoziyoor et al. 2008. *CACTI 5.1*. Technical Report. Technical Report HPL-2008-20, HP Labs.
- [23] Han Xu et al. 2020. Macsen: A processing-in-sensor architecture integrating mac operations into image sensor for ultra-low-power bnn-based intelligent visual perception. *IEEE TCAS II* 68 (2020), 627–631.
- [24] Han Xu et al. 2021. Senputing: An Ultra-Low-Power Always-On Vision Perception Chip Featuring the Deep Fusion of Sensing and Computing. *IEEE TCASI* (2021).
- [25] Tomohiro Yamazaki et al. 2017. 4.9 A 1ms high-speed vision chip with 3D-stacked 140GOPS column-parallel PEs for spatio-temporal image processing. In *ISSCC, IEEE*, 82–83.
- [26] Zheng Zhao et al. 2019. Hardware-software co-design of slimmed optical neural networks. In *ASP-DAC, IEEE*, 705–710.
- [27] Farzaneh Zokaee et al. 2020. LightBulb: A photonic-nonvolatile-memory-based accelerator for binarized convolutional neural networks. In *DATE, IEEE*, 1438–1443.